

Dynamic Programming, Part 1

Fibonacci, Shortest Path in DAGs, LIS, Edit Distance

Recursion + memoization.

Fibonacci

The Sequence

$F_0 = 0$, $F_1 = 1$, and $F_i = F_{i-1} + F_{i-2}$ for $i > 1$.

F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7	...
0	1	1	2	3	5	8	13	

The definition is recursive, so the obvious algorithm is too.

The Obvious Algorithm

```
function fibonacci(i)
  if i == 0
    return 0
  elseif i == 1
    return 1
  else
    return fibonacci(i - 1) + fibonacci(i - 2)
  end
end
```

Its running time satisfies

$$T(n) = T(n-1) + T(n-2) + O(1) \geq 2 T(n-2) + c.$$

Use the recursion tree method to show that `fibonacci` takes exponential time.

Start from $T(n) \geq 2 T(n - 2) + c$.

Unroll the inequality. Each level **doubles** the number of calls while the argument drops by 2:

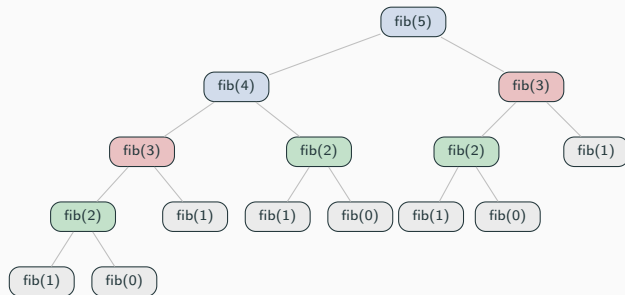
$$T(n) \geq 2 T(n-2) \geq 4 T(n-4) \geq \cdots \geq 2^k T(n-2k).$$

The base case is reached at $k = n/2$, so

$$T(n) \geq 2^{n/2} = \left(\sqrt{2}\right)^n.$$

The exact growth is $\Theta(\varphi^n)$ with $\varphi = \frac{1+\sqrt{5}}{2} \approx 1.618$, but any exponential lower bound makes the point.

Why It Is Wasteful



fib(3) twice, fib(2) three times, and it only gets worse.

Top-Down: Memoization

Before recursing, check whether the answer is already known.

```
function compute_fibonacci(n)
    F = OffsetArray(fill(nothing, n + 1), 0:n)
    F[0] = 0
    F[1] = 1

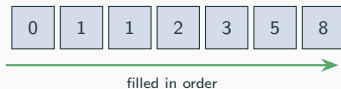
    function fib(i)
        if i == 0 || i == 1
            return F[i]
        else
            if isnothing(F[i - 1]); fib(i - 1); end
            if isnothing(F[i - 2]); fib(i - 2); end
            F[i] = F[i - 1] + F[i - 2]
            return F[i]
        end
    end

    return fib(n)
end
```

Bottom-Up

Or fill the table deliberately, left to right.

```
function fibonacci_bottom_up(n)
    F = OffsetArray(zeros{Int, n + 1}, 0:n)
    F[0] = 0
    F[1] = 1
    for i in 2:n
        F[i] = F[i - 1] + F[i - 2]
    end
    return F[n]
end
```



Exponential becomes $O(n)$.

Shortest Path in DAGs

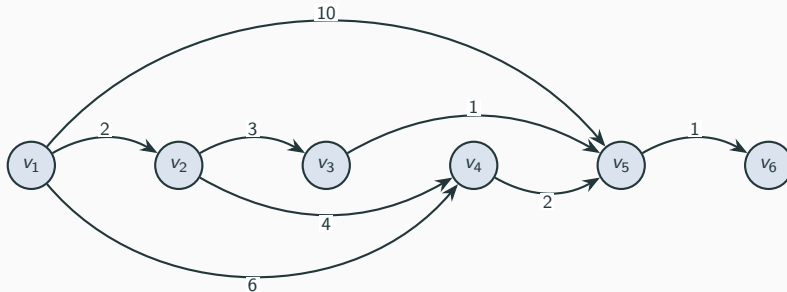
Vertices $v_1, \dots, v_n$ in **topological order**: every edge goes from v_i to v_j with $i < j$.

- $W[v_i, v_j]$ is the length of the edge $v_i \rightarrow v_j$.
- $\text{incoming}(v_i)$ lists the vertices with an edge into v_i .

Such a graph has no cycle. Why?

Goal: compute $\text{dist}[i]$, the shortest path length from v_1 to v_i .

An Example DAG



$\text{incoming}(v_5) = \{v_1, v_3, v_4\}$ with $W[v_1, v_5] = 10$, $W[v_3, v_5] = 1$, $W[v_4, v_5] = 2$.

The Recurrence

- $dist[1] = 0$: the shortest path from v_1 to itself is just v_1 .

The Recurrence

- $dist[1] = 0$: the shortest path from v_1 to itself is just v_1 .
- Any path to v_i arrives through some predecessor v_j , using the edge $v_j \rightarrow v_i$.

The Recurrence

- $dist[1] = 0$: the shortest path from v_1 to itself is just v_1 .
- Any path to v_i arrives through some predecessor v_j , using the edge $v_j \rightarrow v_i$.
- If it passes through v_j , its length is (shortest path to v_j) + $W[v_j, v_i]$.

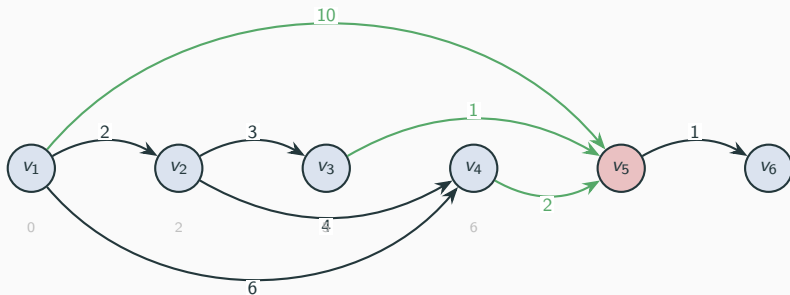
The Recurrence

- $dist[1] = 0$: the shortest path from v_1 to itself is just v_1 .
- Any path to v_i arrives through some predecessor v_j , using the edge $v_j \rightarrow v_i$.
- If it passes through v_j , its length is (shortest path to v_j) + $W[v_j, v_i]$.

$$dist[v_i] = \min_{j < i : v_j \rightarrow v_i \in E} \left(dist[v_j] + W[v_j, v_i] \right)$$

Everything on the right is to the **left** of i , so fill left to right.

Working the Example

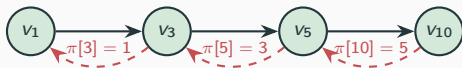


$$\text{dist}[v_5] = \min \left(\underbrace{0 + 10}_{v_1}, \underbrace{5 + 1}_{v_3}, \underbrace{6 + 2}_{v_4} \right) = 6.$$

Reconstructing the Path

If the shortest path to v_i arrives from v_j , then the shortest path to v_j is a **prefix** of it.

So record a predecessor $\pi[i] = j$ each time we improve $dist[i]$, and walk backwards from the target.



Implementation

```
function shortest_path_dag(n, incoming, W)
    dist = fill(Inf, n)
    p = fill(0, n) # p[i] = predecessor of v_i
    dist[1] = 0

    for i in 2:n
        for vj in incoming[i]
            if dist[vj] + W[vj, i] < dist[i]
                dist[i] = dist[vj] + W[vj, i]
                p[i] = vj
            end
        end
    end

    return dist, p
end
```

Running Time

- The outer loop runs $O(|V|)$ times.
- The inner loop walks the incoming edges of v_i .
- Across all i , each edge is examined **exactly once**.

$$O(|V| + |E|)$$

Longest Increasing Subsequence

The Problem

Given $A[1 \dots n]$, find the longest increasing subsequence.

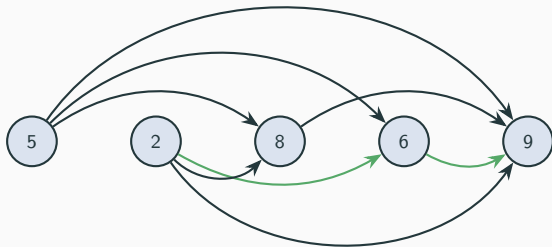


Highlighted: 2, 6, 6, 9 of length 4.

Motivation: one way to measure how sorted a database is. If the LIS is close to n , the data is nearly sorted.

Reduce It to a Longest Path

Build a graph: node i for $A[i]$, and an edge $j \rightarrow i$ of length 1 whenever $j < i$ and $A[j] \leq A[i]$.



An increasing subsequence is exactly a **directed path**.

This graph is a DAG, so we are back to the previous problem.

The Recurrence

Let $L[i]$ be the length of the longest increasing subsequence **ending at** $A[i]$.

- $L[1] = 1$: the path ending at v_1 is just v_1 .
- Otherwise extend the best predecessor, or start fresh.

$$L[i] = \begin{cases} \max_{j < i} (L[j] + 1) & \text{if some } j < i \text{ has } A[j] \leq A[i], \\ 1 & \text{otherwise.} \end{cases}$$

The answer is $\max_i L[i]$, not $L[n]$.

Implementation

```
function lis(A)
    n = length(A)
    L = ones(Int, n) # every entry starts at 1

    for i in 2:n
        for j in 1:i-1
            if A[j] <= A[i] && L[j] + 1 > L[i]
                L[i] = L[j] + 1
            end
        end
    end

    return maximum(L)
end
```

Two nested loops, so $O(n^2)$.

Edit Distance

The Problem

Given strings A and B of lengths n and m : what is the minimum number of **insertions**, **deletions**, and **replacements** that turn A into B ?

Applications: comparing documents, comparing DNA sequences.

Think of it as placing **gaps** to align the two strings.

Two Alignments of the Same Pair

T	R	A	C	E
C	R	A	F	T

3 edits

Two Alignments of the Same Pair

T	R	A	C	E
C	R	A	F	T

3 edits

-	T	R	A	C	E	-
C	-	R	A	-	F	T

5 edits

The edit distance is the alignment with the **fewest** edits.

The Table

Definition

$ED[i, j] =$ edit distance between $A[1 \dots i]$ and $B[1 \dots j]$.

Look at the **last column** of the optimal alignment. There are only three possibilities.



delete



insert



substitute

The Three Cases

- **Delete** $A[i]$: $ED[i - 1, j] + 1$.

Transform $A[1 \dots i - 1]$ into $B[1 \dots j]$, then drop $A[i]$.

The Three Cases

- **Delete** $A[i]$: $ED[i - 1, j] + 1$.
Transform $A[1 \dots i - 1]$ into $B[1 \dots j]$, then drop $A[i]$.
- **Insert** $B[j]$: $ED[i, j - 1] + 1$.
Transform $A[1 \dots i]$ into $B[1 \dots j - 1]$, then append $B[j]$.

The Three Cases

- **Delete** $A[i]$: $ED[i - 1, j] + 1$.
Transform $A[1 \dots i - 1]$ into $B[1 \dots j]$, then drop $A[i]$.
- **Insert** $B[j]$: $ED[i, j - 1] + 1$.
Transform $A[1 \dots i]$ into $B[1 \dots j - 1]$, then append $B[j]$.
- **Substitute**: $ED[i - 1, j - 1] + \text{diff}(i, j)$, where

$$\text{diff}(i, j) = \begin{cases} 1 & \text{if } A[i] \neq B[j], \\ 0 & \text{if } A[i] = B[j]. \end{cases}$$

The Recurrence

$$ED[i, j] = \min \{ ED[i-1, j] + 1, ED[i, j-1] + 1, ED[i-1, j-1] + \text{diff}(i, j) \}$$

Base cases: $ED[i, 0] = i$ and $ED[0, j] = j$. Why?



Implementation

```
function edit_distance(A::String, B::String)
    n, m = length(A), length(B)
    ED = OffsetArray(zeros{Int, n + 1, m + 1}, 0:n, 0:m)

    for i in 0:n; ED[i, 0] = i; end # base cases
    for j in 0:m; ED[0, j] = j; end

    for i in 1:n
        for j in 1:m
            diff = A[i] == B[j] ? 0 : 1
            ED[i, j] = min(
                ED[i - 1, j] + 1, # delete A[i]
                ED[i, j - 1] + 1, # insert B[j]
                ED[i - 1, j - 1] + diff # substitute
            )
        end
    end

    return ED[n, m]
end
```

$O(nm)$ cells, $O(1)$ work each.

Extend `edit_distance` to report the actual edits, not just their number.

Which insertions, deletions, and substitutions turn A into B ?

Same idea as π in the DAG shortest path: remember **which of the three** arguments won.

- Store a choice table, or recompute the comparison while backtracking.
- Start at (n, m) and walk back to $(0, 0)$:
 - from $(i - 1, j)$: delete $A[i]$
 - from $(i, j - 1)$: insert $B[j]$
 - from $(i - 1, j - 1)$: substitute, or match if $A[i] = B[j]$
- Reverse the list at the end.

The walk visits $O(n + m)$ cells, so this costs nothing extra.